מעבדה 2: משפת C למערכת ההפעלה (System Calls & Compilation)

הקדמה ומטרות

במעבדה זו נלמד כיצד תוכנית משתמש מתקשרת עם הליבה (Kernel) של מערכת ההפעלה. נכיר את כלי הפיתוח ב Linux, נבין את תהליך ההידור (Compilation) ונממש פונקציות מערכת בסיסיות באמצעות קריאות מערכת (System Calls) לניהול קבצים.

מטרות המעבדה:

- כתיבה, הידור והרצה של תוכניות C בסביבת Linux באמצעות gcc.
- ניהול תהליך הבנייה של פרויקט בעזרת $Makefile$.
- הבנת ההבדל בין פונקציות ספרייה כמו $printf$ לבין קריאות מערכת כמו $write$.
- שימוש בקריאות המערכת $open, read, write, close$

מבנה הנתונים והקבצים

מצורף למעבדה זו קובץ $\boldsymbol{test.txt}$ קובץ טקסט לצורך בדיקות

עליכם ליצר את קבצי המקור בעצמכם:

1. $\boldsymbol{my_cat.c}$ מימוש פקודת cat
2. **my_reverse.c** הדפסת קובץ מהסוף להתחלה.
3. $\boldsymbol{my_cp.c}$ מימוש פקודת cp
4. $\boldsymbol{my_chmod.c}$ מימוש פקודת $chmod$
5. $\boldsymbol{Makefile}$ קובץ לניהול ההידור של שני הפרויקטים.

דגשים והסברים (למידה עצמאית ודוגמאות)

ניהול פרויקט בעזרת Makefile

ככל שהפרויקטים שלנו יהיו מורכבים יותר, לא נרצה להקליד פקודות הידור ארוכות כמו $gcc\ -Wall\ -g$.. בכל פעם מחדש. ה $Makefile$ הוא סקריפט המנהל עבורנו את תהליך הבנייה.

איך זה עובד ?הקובץ מורכב ממטרות (Targets). לכל מטרה יש תנאי מקדים (Dependencies) ופקודה לביצוע.

מבנה המטרה:

```
target: dependencies
        command
```

שימו לב :השורה של ה $command$ חייבת להתחיל בתו $\boldsymbol{Tab}$ (ולא ברווחים!), אחרת היא לא תזוהה כפקודה ע"י $Linux$.

איך מקמפלים?

1. פותחים טרמינל בתיקייה שבה נמצא ה $Makefile$
2. מקלידים $make$. ה $Shell$ יחפש קובץ בשם זה ויבצע את המטרה הראשונה (all)
3. כדי למחוק את הקבצים שהידרנו, מקלידים $make\ clean$ – שימו לב, לא למחוק את קבצי ה c.

הדגלים $-Wall - g$

הבחירה בדגלים אלו היא קריטית במעבדות של מערכות הפעלה, כי הם הופכים את תהליך הפיתוח ב C מהימור לניתוח מדעי. הנה הסבר על מה כל אחד מהם עושה:

1. **הדגל $-Wall$ (Warning all)**
בניגוד לשפות מודרניות כמו Java או Python , מהדר שפת C (gcc) הוא לעיתים "ותרן" מדי. הוא עלול לתת לכם לכתוב קוד שגוי לוגית מבלי לעצור את ההידור.

- **מה עושה הדגל:** הוא מפעיל כמעט את כל הודעות האזהרה של המהדר. הוא יתריע על דברים כמו:
  - משתנים שהגדרתם ולא השתמשתם בהם.
  - שימוש במשתנה לפני שנתתם לו ערך.
  - חוסר התאמה בין סוגי נתונים (למשל השוואת מספר לכתובת זיכרון).
  - שכחת $return$ בפונקציה שאמורה להחזיר ערך.
- **למה זה חשוב:** במערכות הפעלה, באגים קטנים הופכים מהר מאוד לקריסות מערכת Segmentation Fault. עדיף שהמהדר יצעק עליכם עכשיו מאשר שהתוכנית תקרוס בזמן אמת.

2. **הדגל $-g$ (Debug)**
כאשר אנחנו מהדרים קוד, פקודת gcc הופכת את המילים שכתבנו לשפת מכונה (0 ו-1) שהמעבד מבין. בתהליך הזה, המידע על שמות המשתנים ומספרי השורות הולך לאיבוד.

- **מה עושה הדגל:** הוא אומר למהדר להוסיף מידע "דיבאג" לתוך הקובץ הבינארי שנוצר. המידע הזה יוצר "מפה" שמקשרת בין שורות הקוד שכתבתם לבין הפקודות בזיכרון המחשב.
- **למה זה חשוב:** בלי הדגל הזה, אם התוכנית תקרוס ותנסו להשתמש בכלי כמו **GDB** (ה Debugger של Linux) או לבדוק דליפות זיכרון עם **Valgrind**, הכלים האלו יגידו לכם שהייתה שגיאה, אבל לא יוכלו להגיד לכם **באיזו שורה** היא קרתה או מה היה שם המשתנה שגרם לזה.

דוגמה ל Makefile

```
# הגדרת משתנים לשימוש חוזר
CC = gcc
CFLAGS = -Wall -g

# המטרה הראשונה היא ברירת המחדל (רצים כשכותבים make)
all: my_cat my_info my_cp

# איך בונים את my_cat? הוא תלוי בקובץ המקור my_cat.c
my_cat: my_cat.c
        $(CC) $(CFLAGS) my_cat.c -o my_cat

# מטרה לניקוי קבצים בינאריים
clean:
        rm -f my_cat my_info my_cp
```

קריאות מערכת (System Calls) לניהול קבצים

במערכות הפעלה, כל אינטראקציה עם החומרה (כמו קריאה מהדיסק או כתיבה למסך) חייבת לעבור דרך הליבה (Kernel) אנחנו משתמשים ב – **File Descriptors (FD)** - מספרים שלמים המייצגים את "הצינורות" דרכם זורם המידע.

הפונקציות המרכזיות:

1. ***Open*:** פותחת קובץ ומחזירה FD . אם הערך שחזר הוא 1- הפתיחה נכשלה.
   - דגלים חשובים: O_RDONLY (קריאה בלבד), O_WRONLY (כתיבה בלבד), O_CREAT (צור אם לא קיים).
2. ***read/write*:** מעבירות בתים בין ה – FD לבין חוצץ (Buffer) בזיכרון.
3. ***stat*:** פונקציה מיוחדת שלא פותחת את הקובץ אלא "מציצה" בנתונים שלו (Metadata) כמו גודל והרשאות.
   - $struct\ stat$: מבנה נתונים מובנה ב Linux (מוגדר ב $sys/stat.h$) המכיל מידע על קבצים (גודל, הרשאות, זמן יצירה). נשתמש בשדה st_size (גודל בבתים).

דוגמה לשימוש ב *stat* לקבלת גודל קובץ:

```c
#include <sys/stat.h>
#include <stdio.h>

int main(int argc, char *argv[]) {
    struct stat st; // יצירת משתנה מסוג המבנה stat
    if (argc < 2) return 1;

    // הפונקציה stat ממלאת את המבנה st בנתוני הקובץ
    if (stat(argv[1], &st) == 0) {
        printf("Size of %s is %ld bytes\n", argv[1], st.st_size);
    }
    return 0;
}
```

מהו באמת? File Descriptor (FD)

כשאתם משתמשים ב $fopen$ בשפת C, אתם מקבלים "אובייקט" (מצביע למבנה $FILE$) שמנהל עבורכם זיכרון חוצץ ($BUFFER$). אבל ברמת מערכת ההפעלה, הליבה (Kernel) לא מכירה אובייקטים של C , היא מכירה רק מספרים.

התהליך עובד כך:

1. לכל תהליך (Process) שרץ במערכת יש "טבלת קבצים פתוחים "פרטית בתוך ה Kernel.
2. ה File Descriptor הוא בסך הכל האינדקס (המיקום) בתוך הטבלה הזו.
3. בכל פעם שאתם פותחים קובץ, המערכת מוצאת את השורה הפנויה הראשונה בטבלה, שמה שם מידע על הקובץ, ומחזירה לכם את מספר השורה.

שלושת ה FD הקבועים:

בכל פעם שתהליך חדש נוצר ב Linux , הוא מקבל "בירושה" שלושה קבצים פתוחים כבר במיקומים 0, 1 ו-2:

FD	שם(Standard)	לאן הוא מקושר בברירת מחדל?
0	$stdin$	המקלדת (קלט מהמשתמש)
1	$stdout$	המסך (פלט רגיל)
2	$stderr$	המסך (פלט שגיאות)

דוגמה לכתיבה למסך ללא $printf$ (מימוש הפונקציה my_echo)

```
#include <unistd.h>
#include <string.h>

int main() {
    char *msg = "Hello OS World\n";
    // שימוש ב-System Call write: 1 הוא ה-FD של stdout
    write(1, msg, strlen(msg));
    return 0;
}
```

מעקב אחר קריאות מערכת עם $strace$

כדי לראות אילו קריאות מערכת התוכנית שלכם מבצעת בפועל, הריצו $strace\ ./my_prog$. תוכלו לראות שם את כל ה $write$, $mmap$, $open$, שהמערכת מבצעת "מתחת למכסה המנוע."

מה זה $mmap$ (Memory Map)

הפונקציה הזו "ממפה" (מקשרת) בין כתובת בזיכרון ה *RAM* של התוכנית שלכם לבין משאב חיצוני (בדרך כלל קובץ על הדיסק או זיכרון של המערכת).

ההבדל בין $read$ ל - $mmap$

- **בקריאה ל $read$ (מה שתעשו במעבדה),** ה *Kernel* מעתיק את המידע מהדיסק לתוך *Buffer* (מערך) שאתם הגדרתם בזיכרון. זו פעולת העתקה.
- **ב$mmap$** , המערכת יוצרת קשר ישיר. הכתובת בזיכרון הופכת להיות "חלון" לתוך הקובץ. אם תכתבו לכתובת הזו בזיכרון, המידע ייכתב אוטומטית לקובץ בדיסק מבלי שתצטרכו לקרוא ל $write$.

קריאת המערכת $lseek$ – הזזת סמן הקובץ

בכל פעם שפותחים קובץ, מערכת ההפעלה שומרת "סמן" (*Pointer*) שמציין איפה אנחנו נמצאים בקובץ. כשאנחנו קוראים בית אחד, הסמן זז צעד אחד קדימה. הפונקציה $lseek$ מאפשרת לנו להזיז את הסמן הזה לאן שבא לנו (גישה אקראית).

חתימת הפונקציה:

```
#include <unistd.h>
#include <sys/types.h>

off_t lseek(int fd, off_t offset, int whence);
```

הסבר הארגומנטים:

1. fd **:** המספר של הקובץ (שחזר מהקריאה $open$).
2. $offset$**:** כמה בתים להזיז את הסמן. יכול להיות חיובי (קדימה) או שלילי (אחורה).
3. $whence$**:** מאיזו נקודה להתחיל לספור את ה $offset$. ישנן 3 אפשרויות קבועות:
   - $SEEK_SET$ מתחילת הקובץ (הבית הראשון הוא 0).
   - $SEEK_CUR$ מהמיקום הנוכחי של הסמן.
   - $SEEK_END$ מסוף הקובץ.

דוגמאות לשימוש:

- **הגעה לסוף הקובץ (כדי לדעת מה הגודל שלו)** $off_t\ size\ =\ lseek(fd, 0, SEEK_END)$
- **חזרה לתחילת הקובץ** $lseek(fd, 0, SEEK_SET)$
- **קפיצה תו אחד אחורה מהמיקום הנוכחי** $lseek(fd, -1, SEEK_CUR)$

משימות לביצוע

בכל המשימות המצורפות עליכם לעשות שימוש בקריאות מערכת בלבד. לא ניתן להשתמש בפונקציות ספרייה כמו $printf\ \&\ scanf$

משימה 1: הדפסת קובץ $(my_cat.c)$

ממשו את פקודת cat הבסיסית:

1. קבלת שם קובץ כארגומנט.
2. פתיחת הקובץ במצב $Read\ Only$ אם נכשל, הדפיסו לערוץ השגיאות Error opening file.\n
3. קריאה בלולאה (Buffer של 1024) והדפסה ל $stdout$

משימה 2: הפוך את הסדר $(my_reverse.c)$

דרישות המשימה:

1. התוכנית תקבל שם קובץ כארגומנט.

התוכנית תפתח את הקובץ ותגלה את גודלו בעזרת $stat$ או lseek

התוכנית תדפיס לפלט הסטנדרטי את תוכן הקובץ **בסדר הפוך** (מתו האחרון לתו הראשון)

האתגר : אסור לקרוא את כל הקובץ למערך ענק בזיכרון (הקובץ יכול להיות בגודל 1GB או יותר). עליכם להשתמש ב $lseek$ כדי להזיז את ה – "סמן" של הקובץ תו-תו מהסוף להתחלה, לקרוא כל פעם תו אחד ולהדפיסו.

משימה 3: העתקת קובץ $(my_cp.c)$

ממשו את פקודת cp המעתיקה קובץ מכתובת המקור לכתובת היעד.

1. התוכנית תקבל שני ארגומנטים מקור ויעד.
2. פתיחת המקור לקריאה והיעד לכתיבה.
3. **דגש :**אם היעד לא קיים, צרו אותו עם הרשאות 0644. אם הוא קיים, מחקו את תוכנו (O_TRUNC)
4. בצעו את ההעתקה בעזרת $read\ \&\ write$

משימה 4: שינוי הרשאות תוכניתי $(my_chmod.c)$

במשימה זו נלמד לשנות תכונות של קובץ ישירות מהקוד.

1. התוכנית תקבל שני ארגומנטים: שם קובץ ומחרוזת המייצגת הרשאות בפורמט אוקטלי (למשל 644 או 755).
2. השתמשו בקריאת המערכת $chmod(const\ char\ *path, mode_t\ mode)$.

3. **שימו לב :**כדי להפוך מחרוזת של מספר אוקטלי (כמו "644") לערך שפונקציית $chmod$ מבינה, עליכם להשתמש בפונקציה $strtol("644", NULL, 8)$.
4. אם הפעולה נכשלה, הדפיסו לערוץ השגיאות $\text{Chmod failed.\n}$

משימה 5: כתיבת Makefile

צרו Makefile המכיל:

- מטרה all מהדרת את כל קבצי ה c שכתבתם. (ארבעה קבצים)
- שימוש בדגלים -Wall -g.
- מטרה clean מוחקת את שלושת הקבצים הבינאריים.

מה להגיש ובאיזה פורמט

יש להגיש קובץ ZIP בשם $Lab2_ID.zip$ (החליפו את ID בתעודת הזהות שלכם) על הקובץ להכיל את הקבצים:

1. $my_cat.c$
2. $my_reverse.c$
3. $my_cp.c$
4. $my_chmod.c$
5. $Makefile$
6. $test.txt$

<u>דגשים להגשה:</u>

- יש לוודא שלכל הקבצים יש הרשאות ריצה.(chmod +x)
- הבודק האוטומטי משווה פלטים בצורה מדויקת .(Case Sensitive) אין להדפיס הודעות נוספות מעבר לנדרש.
- וודאו שכל סקריפט מתמודד נכון עם ארגומנטים חסרים.

אי הקפדה על הנחיות, כולל פורמט הגשה הדיגיטלי, תגרור הורדה בציון התרגיל.